

# Отчёт

## Rate Limiter

### **Выполнили:**

Григорьев Тимофей М3100 (Разработка)

Коконов Никита М3100 (Разработка)

Коткин Михаил М3100 (Benchmarks)

Маслов Денис М3101 (Benchmarks)

Крылов Дмитрий М3103 (Аналитика)

## Аннотация

Цель исследования — определить оптимальный алгоритм ограничения частоты запросов для различных сценариев нагрузки на основе метрик (пропускная способность, задержка, точность ограничения, использование памяти, обработка burst-трафика).

Задачи работы:

- Реализация потокобезопасного Rate Limiter, поддерживающего множество клиентов (ключей).
- Реализация трёх алгоритмов: Token Bucket, Leaking Bucket, Sliding Window Log.
- Реализация механизма TTL и фоновой/ленивой очистки старых ключей.
- Разработка нагрузочного тестирования с параметризуемыми паттернами трафика.
- Проведение экспериментальных замеров и анализ результатов.

## 1. Теоретическая часть

В современных высоконагруженных системах критически важно обеспечивать отказоустойчивость и стабильность работы серверов.

**Rate Limiter** (ограничитель частоты запросов) — это программный механизм, контролирующий интенсивность входящего трафика. Он позволяет задавать строгие квоты на количество операций, которые конкретный клиент может выполнить за определённый интервал времени.

Внедрение алгоритмов ограничения скорости решает сразу несколько ключевых задач: предотвращает исчерпание вычислительных ресурсов сервера, защищает инфраструктуру от вредоносных атак (таких как DDoS или перебор паролей) и позволяет справедливо распределять пропускную способность системы между всеми пользователями.

### 1.1. Алгоритмы ограничения частоты запросов

#### Token Bucket (Корзина токенов)

Алгоритм представляет собой «корзину», в которую с постоянной скоростью добавляются токены. При обработке запроса из корзины извлекается необходимое для его выполнения количество токенов (стоимость запроса). Если доступных токенов недостаточно, запрос отбрасывается. Ключевым отличием алгоритма является то, что токены могут накапливаться при простое системы (вплоть до максимальной вместимости корзины).

#### Математическая модель

Пусть  $b$  — максимальная вместимость корзины, а  $r$  — скорость

пополнения токенов (штук в секунду). Доступное количество токенов  $T(t)$  перед обработкой нового запроса в момент времени  $t$  вычисляется как:

$$T(t) = \min(b, T(t_{\text{prev}}) + (t - t_{\text{prev}}) \cdot r)$$

Где  $t_{\text{prev}}$  — время предыдущего пересчета. При поступлении запроса стоимостью  $w$  (где  $w$  — некоторое натуральное число): если  $T(t) \geq w$ , запрос успешно принимается, а количество токенов уменьшается на его стоимость ( $T(t) = T(t) - w$ ). В противном случае запрос отклоняется.

### Преимущества и недостатки

- **Плюсы:** Эффективен по использованию памяти. Главное преимущество — способность обрабатывать кратковременные всплески трафика (bursts), так как накопленные за время простоя токены позволяют мгновенно пропустить пачку запросов (или один «тяжелый» запрос).
- **Минусы:** Если размер корзины ( $b$ ) настроен неправильно, внезапный burst нагрузки может перегрузить целевой сервер, так как алгоритм пропустит эти запросы одновременно.

### Сценарии применения

Широко применяется в **API Gateway** и для ограничения действий пользователя (например, публичные API-интерфейсы). Также существуют вариации с несколькими корзинами для приоритизации трафика на сетевых интерфейсах.

### Leaking Bucket (Протекающее ведро)

Алгоритм обеспечивает интуитивно понятный подход с использованием FIFO-очереди. Поступающие запросы добавляются в конец очереди («ведра»). Через равные промежутки времени первый элемент извлекается и обрабатывается. Если очередь заполнена, новые поступающие запросы отбрасываются (или «утекают»).

### Математическая модель

Пусть  $c$  — максимальный размер очереди, а  $r$  — константная скорость обработки (запросов в секунду). Заполненность очереди  $Q(t)$  перед новым запросом:

$$Q(t) = \max(0, Q(t_{\text{prev}}) - (t - t_{\text{prev}}) \cdot r)$$

При поступлении запроса: если  $Q(t) < c$ , он помещается в очередь. В противном случае — отбрасывается.

## Преимущества и недостатки

- **Плюсы:** Гарантированно сглаживает всплески (даже после простоя) и обрабатывает запросы с абсолютно одинаковой скоростью. Эффективен по памяти (размер очереди ограничен) и легко внедряется на одном балансировщике.
- **Минусы:** При резком увеличении трафика очередь забивается старыми запросами, лишая систему возможности быстро обработать свежие. Нет строгих гарантий по времени обработки (зависит от позиции в очереди). Сложно координировать глобальные лимиты между несколькими балансировщиками.

## Сценарии применения

Оптимален для сглаживания трафика и асинхронных систем (например, очереди сообщений). Хорошо подходит для **защиты от DDoS** на уровне инфраструктуры, так как жестко лимитирует пропускную способность, предотвращая перегрузку бэкенда.

## Sliding Window Log (Скользящий журнал)

Алгоритм предполагает точное отслеживание временных меток (timestamp) каждого запроса пользователя. Записи сохраняются в структуру данных (например, hash set) и сортируются. При каждом новом запросе система удаляет устаревшие метки, вышедшие за пределы скользящего окна, и вычисляет количество оставшихся записей. Если их число превышает лимит, запрос отклоняется.

## Математическая модель

Пусть  $W$  — размер временного окна в секундах, а  $L$  — множество временных меток предыдущих запросов клиента. При поступлении нового запроса в момент  $t$ :

1. Множество очищается от старых запросов:  $L_{\text{new}} = \{x \in L \mid x \geq t - W\}$
2. Если размер  $|L_{\text{new}}| < \text{limit}$ , запрос принимается, а метка  $t$  добавляется в  $L_{\text{new}}$ .

## Преимущества и недостатки

- **Плюсы:** Идеальная точность ограничения. Алгоритм плавно отслеживает время каждого конкретного запроса, благодаря чему не создает искусственных «слепых зон» при переходе между временными интервалами. Поскольку трафик оценивается индивидуально для каждого клиента в любой момент времени, исключаются пиковые всплески нагрузки от одновременных действий множества пользователей.

- **Минусы:** Крайне высокая стоимость вычислений и памяти. Хранение timestamp каждого запроса требует много места. Кроме того, пересчет логга при каждом запросе плохо масштабируется на распределенных кластерах.

## Сценарии применения

Используется для **ограничения действий пользователя**, где критически важна абсолютная точность (например, финансовые транзакции или строгие квоты платных API). Совершенно не подходит для **защиты от DDoS**, так как вычисление логов само по себе может исчерпать ресурсы сервера при масштабной атаке.

## Сводное сравнение алгоритмов

| Алгоритм                  | Преимущества                                                                                               | Недостатки                                                                                            | Использование памяти                  | Лучший сценарий                                        |
|---------------------------|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|---------------------------------------|--------------------------------------------------------|
| <b>Token Bucket</b>       | Эффективен по памяти. Отлично обрабатывает кратковременные всплески (bursts).                              | При неправильной настройке размера корзины всплеск может перегрузить целевой сервер.                  | Низкое (хранятся токены и timestamp)  | API Gateway, публичные квоты                           |
| <b>Leaking Bucket</b>     | Гарантированно сглаживает всплески и обрабатывает запросы с абсолютно одинаковой скоростью.                | Очередь забивается старыми запросами при резких всплесках, нет строгих гарантий по времени обработки. | Низкое (размер очереди ограничен)     | Сглаживание трафика, очереди сообщений, защита от DDoS |
| <b>Sliding Window Log</b> | Идеальная точность лимитов. Плавно учитывает запросы во времени, исключая искусственные всплески нагрузки. | Крайне ресурсоемкий. Требует много памяти для хранения меток и CPU для их пересчета.                  | Высокое (хранится каждый лог запроса) | Финансовые API, строгие ограничения                    |

## 1.2. Паттерны трафика

Для корректной оценки алгоритмов ограничения частоты запросов необходимо проанализировать их поведение в условиях различных профилей нагрузки. В рамках данного исследования рассматриваются пять ключевых паттернов трафика и их соответствие реальным сценариям эксплуатации систем.

### Uniform (Равномерный трафик)

Характеризуется строго константной интенсивностью запросов без выраженных всплесков или падений.

**Реальные сценарии:** Межсерверное взаимодействие (M2M),

непрерывный стриминг телеметрии с IoT-устройств или работа систем внутреннего мониторинга (например, регулярный опрос метрик сервером).

### **Burst (Пиковая нагрузка)**

Представляет собой резкое и кратковременное многократное увеличение объема трафика на фоне низкой базовой нагрузки.

**Реальные сценарии:** Старт крупных онлайн-распродаж, массовая рассылка push-уведомлений клиентам или публикация срочных новостей, вызывающая лавинообразный приток пользователей.

### **Diurnal (Суточный цикл)**

Плавное волнообразное изменение интенсивности трафика, привязанное к времени суток и коррелирующее с периодами бодрствования пользователей.

**Реальные сценарии:** Классические B2C-сервисы (социальные сети, стриминговые платформы), где пик нагрузки приходится на вечерние часы, а глубокий спад — на ночное время.

### **Sparse + Burst (Разреженный со всплесками)**

Большую часть времени система находится в состоянии простоя (единичные запросы или их полное отсутствие), однако в строго определенные моменты генерируется пиковая нагрузка.

**Реальные сценарии:** Пакетная обработка данных (batch processing), запуск тяжелых вычислительных задач по расписанию (cron-скрипты, генерация отчетов в конце дня) или работа систем экстренного оповещения.

### **Zombie (Паразитный трафик)**

Монотонный, непрерывный трафик, не подчиняющийся суточным циклам или бизнес-логике приложения. Часто генерируется в фоновом режиме и игнорирует ответы сервера.

**Реальные сценарии:** Активность вредоносных ботов, скрипты для автоматического парсинга (скрапинга) контента, «осиротевшие» фоновые процессы на устройствах клиентов или медленные DDoS-атаки прикладного уровня.

## **1.3. Параметры системы, задаваемые пользователем**

Для проведения корректного нагрузочного тестирования и эмуляции реальных условий эксплуатации в системе предусмотрена гибкая настройка параметров. Ниже описаны ключевые конфигурационные величины, задаваемые перед началом каждого эксперимента.

## Лимиты (Rate и Burst Capacity)

Базовые ограничения пропускной способности. Параметр rate определяет целевую скорость обработки (количество разрешенных запросов в секунду). Параметр burst\_capacity (применимо для Token Bucket) задает размер «окна толерантности» — максимальное количество запросов, которое система готова пропустить одномоментно при наличии накопленной квоты.

**Влияние:** Данные параметры напрямую определяют строгость алгоритма и его способность сглаживать микро-всплески легитимного трафика.

## TTL ключей (Time-to-Live)

Время жизни состояния неактивного клиента в памяти системы. Если от конкретного ключа не поступает запросов в течение заданного интервала TTL, его данные помечаются как устаревшие и удаляются механизмом фоновой очистки.

**Влияние:** Критически важный параметр для предотвращения исчерпания памяти при обработке бесконечного потока разовых посетителей или активности вредоносных ботов (паттерн Zombie).

## Длительность тестирования

Временной интервал, в течение которого генератор нагрузки отправляет запросы в систему согласно выбранному паттерну.

**Влияние:** Длительность должна быть достаточной для выхода системы на стабильный режим работы. Это позволяет нивелировать погрешности от начального «прогрева» кэшей процессора и аллокатора памяти, а также дает время на срабатывание фоновых процессов очистки устаревших ключей.

## Количество потоков-клиентов

Число параллельных рабочих потоков, генерирующих запросы от лица множества уникальных ключей.

**Влияние:** Задает уровень конкурентного доступа. Позволяет проверить эффективность шардирования, оценить накладные расходы на ожидание разблокировки мьютексов и доказать отсутствие состояний гонки данных при пиковых нагрузках.

## Количество повторных запусков

Число итераций одного и того же эксперимента с последующим вычислением медианных и средних значений результатов.

**Влияние:** Гарантирует статистическую значимость исследования. Повторные запуски позволяют исключить влияние случайных факторов

среды исполнения, таких как прерывания планировщика ОС или активность фоновых системных процессов.

## **1.4. Метрики оценки экспериментов**

Для объективного сравнения реализованных алгоритмов в условиях различных профилей нагрузки используется следующий набор метрик:

### **Accuracy (Точность ограничения)**

Измеряется как процентное отклонение реально пропущенных запросов от заданного лимита. Данная метрика критически важна для подтверждения корректности работы механизмов синхронизации. Она доказывает успешное решение проблемы «check-then-act» при конкурентном доступе множества потоков к счётчикам одного ключа и гарантирует отсутствие гонок данных.

### **Throughput (Пропускная способность)**

Измеряется в количестве запросов в секунду (RPS), которые сам Rate Limiter способен обработать в чистом виде (без учета сетевых задержек). Характеризует процессорные накладные расходы алгоритма на захват мьютексов, работу с атомиками и хеш-таблицами.

### **p50 / p99 latency (Перцентили задержки)**

Измеряется в микросекундах или наносекундах. Значение p50 показывает базовую скорость проверки лимита (когда нет жесткой конкуренции потоков). Значение p99 иллюстрирует задержку для 1% худших случаев — например, когда поток вынужден ожидать блокировки из-за срабатывания фоновой очистки или реаллокации памяти в структурах данных.

### **Memory usage (Использование памяти)**

Измеряется в байтах на каждого уникального клиента (ключ). Отражает объем памяти, занимаемый внутренним состоянием: от двух базовых переменных в алгоритме Token Bucket до динамических структур для хранения временных меток в Sliding Window Log.

### **Burst handling (Обработка всплесков)**

Оценивает поведение алгоритма при поступлении пиковой нагрузки. Фиксирует, какое количество запросов сверх стандартного лимита алгоритм пропускает единоразово (в пределах параметра burst\_capacity), и насколько быстро он переходит в режим блокировки после исчерпания емкости.

### **Eviction count (Количество очисток)**

Абсолютное число неактивных ключей, удаленных из хеш-таблицы

механизмом ленивой очистки или фоновым потоком по истечении параметра TTL. Метрика доказывает отсутствие утечек памяти и эффективность управления жизненным циклом клиентов.

## 1.5. Потокобезопасность и гонки данных

В высоконагруженных системах запросы от одного и того же клиента могут обрабатываться параллельно пулом рабочих потоков сервера. Это неизбежно приводит к конкурентному доступу к внутреннему состоянию алгоритма ограничения (счётчикам токенов, логам времени).

### Проблема «Check-then-act» (Проверь, затем сделай)

Классическая уязвимость многопоточной среды. Возникает, когда поток успешно проверяет наличие квоты (check), но перед фактическим списанием токена (act) планировщик ОС прерывает его работу. В этот момент другой поток также проходит проверку и списывает ту же самую квоту. По возвращении первый поток завершает начатое действие, уводя счетчик в отрицательное значение. В результате система пропускает больше запросов, нарушая метрику Assurasy.

### Ограничения `std::atomic`

Хотя использование `std::atomic` защищает отдельные переменные от повреждения, в контексте алгоритмов Rate Limiter этого подхода зачастую недостаточно. Большинству алгоритмов требуется одновременное обновление сразу нескольких полей — например, и счетчика токенов, и метки времени. Гарантировать атомарность таких составных операций через сложные Compare-And-Swap циклы крайне ресурсоемко и ведет к запутанному коду.

### Блокировки на уровне ключа (Mutex per key)

Надежный подход, при котором состояние каждого клиента защищается собственным примитивом синхронизации (например, `std::mutex`). Вся логика метода `Access` оборачивается в критическую секцию (`std::lock_guard`), что полностью исключает проблему «check-then-act». Основной недостаток — избыточное потребление памяти (один мьютекс на каждого клиента) и разрозненность объектов в куче, что ухудшает локальность данных.

### Шардирование ключей (Sharded maps) и оптимизация памяти

Для баланса между потокобезопасностью и пропускной способностью применяется разбиение хранилища на фиксированное количество шардов. Каждый шард отвечает за свою группу ключей и защищается одним общим мьютексом. Запросы к разным шардам выполняются

абсолютно параллельно.

**Устранение False Sharing:** При плотной упаковке шардов в памяти возникает аппаратная проблема «ложного разделения». Независимые потоки, обращаясь к разным мьютексам, могут инвалидировать кэш друг друга, если эти данные лежат в одной кэш-линии процессора. Для решения этой проблемы в высокопроизводительных Rate Limiter'ах применяется выравнивание памяти (placement new и кастомные аллокаторы), гарантирующее, что данные разных шардов аппаратно разнесены блоками по границам кэш-линий.

## 2. Архитектура и реализация программного решения

Разработка библиотеки `avito_limiter` выполнена на языке C++23 с активным использованием возможностей стандартной библиотеки, шаблонного метапрограммирования (C++ Concepts) и идиомы CRTP. Архитектура построена по модульному принципу, строго разделяя ответственность между алгоритмами ограничения, хранилищем состояния и механизмами потокобезопасности.

### 2.1. Базовые интерфейсы и диспетчеризация

Взаимодействие с механизмами управления трафиком абстрагировано через два ключевых полиморфных интерфейса, решающих разные задачи:

#### **IRateLimiter**

Синхронный интерфейс для жесткого ограничения запросов. Принимает строковый ключ (`key_type = std::string`) и предоставляет методы `Exists(key)`, `Access(key)` и `GetNumAvail(key)`. Решение о пропуске или блокировке запроса принимается системой атомарно и незамедлительно. Состояние каждого ключа полностью изолировано.

#### **IRateShaper**

Асинхронный интерфейс для формирования и сглаживания (traffic shaping) поступающего потока. Метод `AddRequest(key)` возвращает `std::optional<std::future<bool>>`. Если внутренняя очередь не переполнена, запрос переходит в ожидание, а клиентский код получает `future`, который асинхронно перейдет в состояние `true` в момент фактического пропуска запроса.

Связывание интерфейсов с конкретными реализациями хранилищ и алгоритмов на этапе компиляции осуществляется через класс `RateLimiterWrapper`. Для работы в высоконагруженных сценариях

предусмотрена обертка `ShardedWrapper`, которая горизонтально масштабирует систему, шардируя ключи по независимым экземплярам лимитеров (используя вычисление `hash(key) % CountShards`).

## 2.2. Реализация алгоритмов

Ядро вычислительной логики составляют три алгоритма. Первые два оформлены как `header-only` шаблоны, не зависящие от ключей или механизмов синхронизации, что делает их легко тестируемыми и переиспользуемыми:

### Token Bucket (`TokenBucketAlgo`)

Реализует логику «корзины токенов» с заданными параметрами максимальной вместимости (`capacity`) и скорости пополнения (`refill_tok_sec`). Внутренний метод `Update()` непрерывно пересчитывает доступные токены с учетом прошедшего времени. Метод `Access()` списывает токены по мере поступления запросов.

### Sliding Window Log (`SlidingWindowAlgo`)

Обеспечивает точный лог-ориентированный учет в скользящем окне без привязки к дискретным интервалам. Алгоритм сохраняет временные метки (`timestamps`) успешных запросов в структуру на основе `min-heap` и динамически вытесняет устаревшие записи при каждом новом обращении.

### Leaky Bucket Shaper (`LeakyBucketShaper`)

Реализует интерфейс `IRateShaper`, инкапсулируя в себе очередь ожидания и независимый фоновый рабочий поток (`RunQueueThread`). Этот поток периодически «пробуждается» и извлекает из очереди строго заданное число запросов, закрывая связанные с ними объекты `std::promise`.

## 2.3. Механизм TTL и управление хранилищем

Для очистки базы данных от устаревших ключей и более эффективного использования памяти реализована комплексная система управления временем жизни (TTL):

- **Оболочка `StoredData`:** Используя идиому CRTP, базовые алгоритмы оборачиваются в структуру, содержащую счетчик времени `TtlValue<Clock>`. Проверки `IsExpired()` и `Prolong()` встроены непосредственно в метод `Access()` алгоритма, что минимизирует накладные расходы на виртуальные вызовы при каждом запросе.
- **Контейнер `MutexStorage`:** Отвечает за хранение пар «ключ — данные» в `std::unordered_map`. Обеспечивает потокобезопасность с помощью

двухуровневой блокировки: `std::shared_mutex` на всю хеш-таблицу и опционального `std::mutex` на каждый отдельный ключ для защиты от состояния гонки (data race).

Очистка устаревших записей в `MutexStorage` выполняется двумя взаимодополняющими стратегиями. **Ленивая очистка** производит атомарную проверку прошедшего интервала при каждом обращении к хранилищу и удаляет ключи в вызывающем потоке. **Фоновая очистка** опирается на выделенный поток (`RunCleaner`), который регулярно сканирует всю таблицу в эксклюзивном режиме.

## 2.4. Модульное тестирование

Верификация корректности работы всех компонентов обеспечивается комплексом юнит-тестов на базе фреймворка Google Test.

Детерминированность тестов достигается за счет использования инжектируемых «искусственных» часов (`MockClock`). Покрывание сфокусировано на четырех критических аспектах:

- **Точность лимита:** Валидация математических моделей алгоритмов (корректность пополнения бакета и вытеснения логов) на различных временных интервалах.
- **Burst-обработка:** Оценка поведения системы при пиковых всплесках трафика и контроль параметра `saracity`.
- **Потокобезопасность:** Симуляция жесткой конкуренции при параллельных вызовах к одному ключу для доказательства отсутствия состояния гонки (data race).
- **Жизненный цикл (Eviction):** Эмуляция течения времени для проверки корректного и своевременного срабатывания механизмов удаления устаревших ключей.

## 2.5. Нагрузочное тестирование

Для проведения экспериментального анализа производительности реализован параметризуемый многопоточный генератор трафика, интегрированный с инфраструктурой Google Benchmark. Инструмент поддерживает точную эмуляцию пяти ключевых паттернов нагрузки: `Uniform` (равномерный), `Burst` (всплески), `Diurnal` (суточные циклы), `Sparse + Burst` (разреженный со всплесками) и `Zombie` (паразитный фоновый трафик).

Генератор позволяет гибко конфигурировать интенсивность поступающих запросов, количество потоков-клиентов и параметры распределения ключей. Это дает возможность собирать репрезентативные метрики (Throughput, p50/

p99 latency) и формировать надежную статистическую базу для сравнения эффективности алгоритмов в реальных условиях.

### 3. Экспериментальная часть

**Общие настройки:** MinTime = 10 с, Repetitions = 3, Threads = 8.

Замерялись p50 и p99 latency вызова Access() (mean по повторам).

#### Гипотеза 1

**Суть:** При высокой конкурентной нагрузке ShardedWinLimiter даёт меньший p99, чем MutexWinLimiter.

**Прогон:** Uniform, 1000 RPS. Mutex — один hot key; Sharded — cross-shard (2 hot keys на границе шардов).

| Реализация           | p50, нс | p99, нс     | Δ p50  | Δ p99  |
|----------------------|---------|-------------|--------|--------|
| MutexWinLimiter      | 1 833   | 60 600 ±8%  | —      | —      |
| ShardedWin (compact) | 1 733   | 51 500 ±11% | −5.5%  | −15.0% |
| ShardedWin (aligned) | 1 600   | 39 467 ±14% | −12.7% | −34.9% |

**Вывод:** Sharded снижает p99: compact — на 15%, aligned — на 35%. Aligned-версия эффективнее за счёт выравнивания шардов по кэш-линии (меньше false sharing). Гипотеза **подтверждена**.

#### Гипотеза 2

**Суть:** На Zombie-трафике Sliding Window хуже Token Bucket по latency (и ожидается, что по памяти).

**Прогон:** Zombie, 5000 RPS, 2 запроса на ключ, TTL = 5 с, lazy cleaner каждые 1 с.

| Реализация     | p50, нс | p99, нс         | Δ p50 | Δ p99 | Память           |
|----------------|---------|-----------------|-------|-------|------------------|
| Token Bucket   | 1 100   | 107 533<br>±19% | —     | —     | —                |
| Sliding Window | 2 700   | 194 367<br>±33% | +145% | +81%  | не<br>измерялась |

**Вывод:** Sliding Window работает значительно медленнее: p50 в 2.5 раза выше, p99 — на 81%. Token Bucket хранит  $O(1)$  состояние на ключ, в то время как Window сохраняет timestamp на каждый запрос и требует работы с кучей при ротации ключей. Потребление памяти бенчмарком не оценивалось. Гипотеза **частично подтверждена** (по latency — да, по памяти — нет данных).

### Гипотеза 3

**Суть:** На Burst-нагрузке Token Bucket быстрее пропускает легальный всплеск с меньшей задержкой.

**Прогон:** Burst: 50 запросов  $\times$  100 мс, пауза 1 с, burst\_capacity = 100, 8 потоков.

| Реализация     | p50, нс | p99, нс          | $\Delta$ p50 | $\Delta$ p99 |
|----------------|---------|------------------|--------------|--------------|
| Token Bucket   | 1 167   | 33 267 $\pm$ 48% | —            | —            |
| Sliding Window | 1 500   | 56 667 $\pm$ 92% | +29%         | +70%         |

**Вывод:** Token Bucket показывает p50  $\approx$  1.2 мс, p99  $\approx$  33 мс; Sliding Window хуже на 29% и 70% соответственно. На всплеске Bucket списывает токен за  $O(1)$ , тогда как Window очищает журнал на каждый запрос. (Метрика items\_per\_sec здесь не учитывается, так как на неё влияет sleep между итерациями). Гипотеза **подтверждена**.

### Сводка результатов

| № | Гипотеза                           | Итог                             |
|---|------------------------------------|----------------------------------|
| 1 | Sharded p99 < Mutex p99 @ 1000 RPS | <b>подтверждена</b>              |
| 2 | Window хуже Token на Zombie        | <b>частично<br/>подтверждена</b> |
| 3 | Token быстрее на Burst             | <b>подтверждена</b>              |

## 4. Результаты работы